

Stage 2 of 5

Zero-Parameter SLIC Supervoxel Generation

Comprehensive Technical Documentation

`supervoxel_generator.py`

Institution CMC Vellore	Laboratory QIRAIL	Dataset 120 HNSCC	Reference Bae et al., 2025
-----------------------------------	-----------------------------	-----------------------------	--------------------------------------

Table of Contents

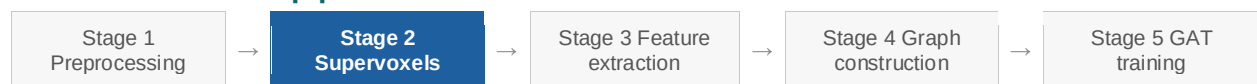
1.1 What this stage does.....	3
1.2 Position in the pipeline.....	3
1.3 Key design principle	3
2.1 The problem with whole-tumour radiomics.....	3
2.2 Approach comparison.....	3
4.1 Code.....	4
4.2 Why this is the most important design decision	4
5.1 Code	5
5.2 Equation.....	5
5.3 Why region-restricted normalisation matters	5
6.1 Initialise cluster centres on a regular grid	6
6.2 Standard SLIC distance metric	6
6.3 SLIC-zero — adaptive compactness (the zero-parameter innovation)	7
6.4 Local search window — why SLIC is $O(N)$	7
6.5 Centre update	8
6.6 Convergence check.....	8
6.7 enforce_connectivity — post-iteration cleanup	8
7.1 Apply region mask.....	8
7.2 Renumber labels to contiguous integers	8
7.3 Sanity check.....	9
10.1 Upstream — preprocessing.py	10
10.2 Downstream — feature_extractor.py.....	10
10.3 Downstream — graph_builder.py.....	10

1 Overview

1.1 What this stage does

Supervoxel generation partitions the 3D **peritumoral region** surrounding the gross tumour volume (GTV) into a set of compact, spatially contiguous sub-volumes called **supervoxels**. Each supervoxel becomes a node in the patient-level graph later fed into the Graph Attention Network (GAT). The algorithm used is **Simple Linear Iterative Clustering (SLIC)**, running in zero-parameter mode (`slic_zero=True`) — a setting central enough to appear in the paper title.

1.2 Position in the pipeline



1.3 Key design principle

Note

The GTV is not segmented into supervoxels. It is excluded before SLIC runs and treated as a separate hub node in the graph. Supervoxels cover only the surrounding peritumoral tissue.

2 Background — Why Supervoxels?

2.1 The problem with whole-tumour radiomics

Traditional radiomics extracts one feature vector per patient from the entire GTV. This collapses all spatial information — a region at the invasive tumour margin cannot be distinguished from the tumour core, and the peritumoral tissue is ignored entirely. The RadGraph paper argues that the peritumoral microenvironment carries independent prognostic information about locoregional recurrence.

Supervoxels solve this by dividing the peritumoral tissue into approximately 100 small, locally homogeneous sub-volumes. Each gets its own radiomic feature vector. The spatial relationships between these supervoxels and the GTV are then modelled as a graph.

2.2 Approach comparison

Approach	Spatial detail	Nodes in graph	Role in pipeline
Whole-GTV radiomics	None	1	Baseline RF model
Individual voxels	Maximum	~10 ⁷	Computationally infeasible
SLIC supervoxels	High	~100	RadGraph — this pipeline

3 Inputs

The function accepts three 3D numpy arrays of identical spatial dimensions, plus configuration constants from config.py.

Parameter	Shape	dtype	Description
<code>ct_array</code>	(Z, H, W)	float32	Resampled CT in Hounsfield units, 1 mm isotropic spacing
<code>region_mask</code>	(Z, H, W)	uint8	Binary mask of the 5 cm peritumoral bounding region (1 = inside)
<code>gtv_array</code>	(Z, H, W)	uint8	Binary GTV mask. Excluded from SLIC before running (see Section 4)
<code>n_segments</code>	scalar	int	Target supervoxel count, default 100 (from config.SLIC N SEGMENTS)

4 Step 1 — GTV Exclusion from the SLIC Region

Before any segmentation begins, the GTV voxels are **zeroed out in the peritumoral mask**. SLIC will never see or label those voxels. This creates an annular *peritumoral-only* region that SLIC segments, while the GTV remains untouched.

4.1 Code

```

Python
# Step 1: copy region mask and remove GTV voxels
peritumoral_mask = region_mask.copy()
if gtv_array is not None:
    peritumoral_mask[ gtv_array > 0 ] = 0    # GTV voxels zeroed out

# SLIC receives only the peritumoral mask
labels = slic(ct_normalized, mask=peritumoral_mask, slic_zero=True, ...)

# Consequence: all GTV voxels have label = 0 (background)

```

4.2 Why this is the most important design decision

- **Graph topology:** The GTV is always node 0 (the hub) in the star graph. If SLIC fragments the GTV into 3–5 sub-regions, those fragments become multiple spoke nodes — breaking the intended hub-and-spoke structure.
- **Signal purity:** Supervoxels are intended to represent peritumoral microenvironment. Tumour voxels have a qualitatively different radiomic signature. Mixing them into the supervoxel pool contaminates the graph signal.

- **Feature extraction:** In `feature_extractor.py`, GTV features are extracted separately using the raw binary GTV mask — not from the supervoxel label map. The two paths must not overlap.

Tip

The result is an annular region: the 5 cm peritumoral shell with a GTV-shaped hole in the middle. SLIC carves this shell into ~100 supervoxels without crossing the tumour boundary.

5 Step 2 — CT Intensity Normalisation

SLIC clusters voxels based on intensity similarity. Raw Hounsfield Units (HU) range from −1000 (air) to +3000 (dense bone). Feeding this raw range to SLIC would cause it to cluster primarily around extreme air/bone boundaries rather than the clinically relevant soft-tissue transitions. Normalisation is required.

5.1 Code

```
Python
# Compute min and max ONLY within the peritumoral region
ct_in_region = ct_array[ peritumoral_mask > 0 ]
region_min   = ct_in_region.min()
region_max   = ct_in_region.max()

# Min-max normalise
ct_normalized = (ct_array - region_min) / (region_max - region_min + 1e-6)

# Clip to [0, 1] — excludes out-of-region voxels
ct_normalized = np.clip(ct_normalized, 0.0, 1.0)
```

5.2 Equation

Min-max normalisation

$$I_{\text{norm}}(\mathbf{v}) = [I(\mathbf{v}) - \min_R] / [\max_R - \min_R + \epsilon]$$

$I(\mathbf{v})$ — raw HU value at voxel $\mathbf{v}$
 $\min_R$ — minimum HU value computed inside the peritumoral mask R only
 $\max_R$ — maximum HU value computed inside the peritumoral mask R only
 ϵ — small constant ($1e-6$) to prevent division by zero
 $I_{\text{norm}}(\mathbf{v})$ — normalised intensity in $[0, 1]$

5.3 Why region-restricted normalisation matters

If the min/max were computed over the full CT volume, background air voxels at −1000 HU would anchor the minimum. The denominator would be ~4000 HU, compressing the entire peritumoral tissue range (~200–600 HU) into a narrow band of about 0.30–0.40 in the normalised space. SLIC would see almost no intensity variation within the region.

With region-restricted normalisation, the tissue of interest spans the full $[0, 1]$ range, giving SLIC maximum discriminative power to detect intensity boundaries within the peritumoral shell.

Strategy	Denominator	Tissue range in [0, 1]	Outcome
Full-volume normalisation	~4000 HU	0.30 – 0.40 only	Poor tissue contrast
Region-restricted (ours)	~200–600 HU	0.0 – 1.0 (full range)	Maximum tissue contrast

6 Step 3 — The Zero-Parameter SLIC Algorithm

SLIC (Achanta et al., IEEE TPAMI, 2012) generates supervoxels by running a modified **k-means clustering** in a combined intensity–spatial feature space. Two innovations distinguish it from standard k-means: a **local search window** that gives $O(N)$ complexity, and — in zero-parameter mode — an **adaptive compactness** parameter that requires no manual tuning.

6.1 Initialise cluster centres on a regular grid

Before any iteration, SLIC places seed cluster centres on a regular 3D grid within the masked region. The grid spacing S is chosen so that K seeds fit the available volume:

Grid spacing
$S = (V / K)^{(1/3)}$
V — total number of voxels inside the peritumoral mask K — target supervoxel count (default: 100) S — grid spacing in voxels — e.g. for $V = 500,000$ and $K = 100$, $S \approx 17$ voxels (17 mm at 1 mm isotropic)

Seeds that fall outside the peritumoral mask after grid placement are discarded. The result is approximately K seed points distributed throughout the region.

Tip

Think of pressing 100 pins evenly into the peritumoral annular region. Each pin marks the starting centre of one supervoxel cluster. Pins that land on the GTV or outside the region are removed before any iteration begins.

6.2 Standard SLIC distance metric

Standard SLIC balances intensity similarity and spatial proximity using a **fixed compactness parameter m** :

Standard SLIC distance
$D = \sqrt{d_I^2 + (m / S)^2 \times d_S^2}$
d_I — intensity distance = $ I(\text{voxel}) - I(\text{centre}) $ d_S — spatial distance = $\sqrt{(\Delta z^2 + \Delta y^2 + \Delta x^2)}$ m — compactness parameter — fixed global value (higher m = more spherical clusters) S — grid spacing (normalises spatial distance to the cluster scale)

The ratio m / s controls the trade-off: large m produces compact, sphere-like supervoxels; small m produces irregular supervoxels that follow intensity edges. The fundamental problem is that a *single global m cannot suit all tissue types simultaneously* — fat, muscle, and tumour margin each have different optimal compactness.

6.3 SLIC-zero — adaptive compactness (the zero-parameter innovation)

SLIC-zero eliminates the fixed m parameter entirely. Instead, the compactness is computed **per cluster, per iteration**:

Adaptive compactness per cluster	
$m_k = \max\{ d_I(v, c_k) : v \in \text{cluster } k \}$	
m_k	compactness for cluster k — equals the maximum intensity variation observed within cluster k
$d_I(v, c_k)$	intensity distance between voxel v and cluster centre c_k
SLIC-zero distance (replaces standard SLIC distance)	
$D_k = \sqrt{d_I^2 + (m_k / s)^2 \times d_s^2}$	
m_k	recomputed every iteration as cluster membership evolves
D_k	cluster-specific distance — different clusters may have very different m_k

The effect is intuitive: in a homogeneous region such as fat, all voxels within a cluster have similar intensity, so m_k is small, and the distance formula weights **spatial proximity heavily** — producing compact, sphere-like supervoxels. In a heterogeneous region such as a tumour margin, m_k is large, and the formula weights **intensity similarity more** — producing irregular supervoxels that follow tissue boundaries. No parameter tuning is needed. This is why the algorithm is called **zero-parameter**.

Component	Formula	Meaning
Intensity distance d_I	$ I(v) - I(c_k) $	How different this voxel's HU is from its cluster centre
Spatial distance d_s	$\sqrt{(\Delta z)^2 + (\Delta y)^2 + (\Delta x)^2}$	Physical voxel distance between voxel v and cluster centre c_k
Adaptive weight m_k	$\max(d_I)$ within cluster k	Scales with local intensity variance — larger in heterogeneous regions
Combined distance D_k	$\sqrt{d_I^2 + (m_k/s)^2 \times d_s^2}$	Final distance used to assign each voxel to a cluster

6.4 Local search window — why SLIC is $O(N)$

For each cluster centre, SLIC searches only a **$2S \times 2S \times 2S$ neighbourhood** rather than the full volume. Every voxel in this window is assigned to the cluster centre with minimum D_k .

Note

Complexity advantage

Standard k-means: $O(N \times K) = 500,000 \times 100 = 50,000,000$ distance computations per iteration
 SLIC local search: $O(N) = 500,000$ total — each voxel is compared to at most a few nearby centres
 SLIC is approximately 100× faster than k-means for this problem size.

6.5 Centre update

After all voxels are assigned, each cluster centre is recomputed as the mean position and intensity of all its members:

Centre update

$$\mathbf{c}_k = \text{mean}\{ (\mathbf{z}, \mathbf{y}, \mathbf{x}, \mathbf{I}) : \text{label}(\mathbf{v}) = k \}$$

$\mathbf{c}_k$ — new centre for cluster k — moves to the geometric and intensity centroid of all its assigned voxels

6.6 Convergence check

Steps 6.4 and 6.5 repeat until the total displacement of all cluster centres falls below a threshold, or until the iteration limit is reached:

Convergence criterion

$$\mathbf{E} = \sum_k || \mathbf{c}_k(t) - \mathbf{c}_k(t-1) || < \epsilon$$

$\mathbf{E}$ — total residual error — sum of centre displacements across all K clusters

ϵ — convergence threshold (scikit-image default)

t — iteration index — SLIC typically converges in 5–10 iterations for CT data

6.7 enforce_connectivity — post-iteration cleanup

After convergence, SLIC may have produced small disconnected fragments — voxels stranded from their assigned cluster. Setting `enforce_connectivity=True` forces these to merge with the nearest large contiguous neighbour. This guarantees every supervoxel in the output is a **single spatially connected 3D region**, which is essential for reliable radiomic feature extraction.

7

Step 4 — Post-Processing the Label Map

7.1 Apply region mask

Any labels that fall marginally outside the peritumoral boundary after SLIC are set to zero:

Python

```
labels = labels * peritumoral_mask
```

7.2 Renumber labels to contiguous integers

After masking, the label set may contain gaps (for example, labels 1, 2, 5, 9 if some clusters were entirely outside the mask). The `_renumber_labels()` method remaps all surviving labels to 1, 2, 3, ..., N :

Python

```
unique_labels = np.unique(labels)
unique_labels = unique_labels[ unique_labels > 0 ] # exclude background
```



```
renumbered = np.zeros_like(labels)
for new_label, old_label in enumerate(unique_labels, start=1):
    renumbered[ labels == old_label ] = new_label
```

This ensures `range(1, n_supervoxels + 1)` in `feature_extractor.py` covers every valid supervoxel ID with no gaps.

7.3 Sanity check

If the final supervoxel count falls below 20, a warning is printed. This typically indicates either a very small peritumoral region, a very large GTV relative to the region, or coarse input resolution. Downstream code uses `K = min(20, n_supervoxels)`, so the pipeline continues but the patient graph will have fewer spoke nodes.

Caution

Low supervoxel count

If you see this warning frequently across your cohort, check: (1) CT spacing after resampling — should be 1 mm isotropic (2) peritumoral region size — should contain at least 2,000 voxels (3) GTV exclusion — a very large GTV may consume most of the 5 cm region

8 Output Format

Field	Type / Shape	Value semantics
<code>supervoxel_labels</code>	<code>int32 · (Z, H, W)</code>	0 = background / GTV / outside region; 1...N = peritumoral supervoxel IDs
<code>n_supervoxels</code>	<code>int</code> (Python native)	Actual supervoxel count. Typical range: 70–110 (target 100). Stored as <code>np.array([N])</code> in <code>.npz</code>
Saved key	<code>.npz</code> archive	'supervoxel_labels' in {patient_id} <code>preprocessed.npz</code>
Labels are	contiguous	After renumbering: no gaps. Iterating <code>range(1, N+1)</code> reaches every valid supervoxel.

9 Parameter Reference

Parameter	Default	Effect	Change if...
<code>n_segments</code>	100	Target supervoxel count	Region very large (↑ to 150); region very small (↓ to 50–60)
<code>slic_zero</code>	True	Adaptive compactness — zero-parameter mode	Never change. Core to the algorithm's design.

Parameter	Default	Effect	Change if...
<code>channel_axis</code>	None	Grayscale 3D input (no colour channels)	Never change. CT is always single-channel.
<code>enforce_connectivity</code>	True	Merge disconnected fragments	Set False only for debugging. Always True in production.
<code>start_label</code>	1	First supervoxel label (0 = background)	Never change. Convention is fixed across the pipeline.
<code>mask</code>	<code>peritumoral_mask</code>	Restricts SLIC to valid region	Always pass the GTV-excluded peritumoral mask.
Min voxels per SV	8	Skip tiny supervoxels in feature extraction	Reduce to 4 if many boundary supervoxels are being dropped.

10 Integration with the Rest of the Pipeline

10.1 Upstream — preprocessing.py

`preprocessing.py` resamples the CT and GTV mask to 1 mm isotropic spacing and calls `define_peritumoral_region()` to define the 5 cm bounding region. The three arrays `ct_array`, `gtv_array`, and `region_array` are all required inputs to Stage 2.

10.2 Downstream — feature_extractor.py

`feature_extractor.py` iterates `range(1, n_supervoxels + 1)` and for each label ID:

1. Creates a binary mask: `sv_mask = (supervoxel_labels == sv_id)`
2. Skips if fewer than 8 voxels (too small for reliable radiomics)
3. Calls PyRadiomics to extract ~83 texture and intensity features
4. Records the voxel centroid (z, y, x) as a spatial coordinate for the graph

GTV features are extracted **separately** using the binary `gtv_array` — not from the supervoxel label map. This is why the GTV exclusion in Step 1 is essential: the two feature extraction paths must not overlap.

10.3 Downstream — graph_builder.py

`graph_builder.py` receives the per-supervoxel feature vectors and centroids and constructs the patient graph:

- **Node 0 (hub):** GTV features and centroid
- **Nodes 1–20 (spokes):** Top-20 supervoxels selected by cosine similarity to GTV features
- **Edge features:** 3D Euclidean distance between node centroids in physical space (mm)
- **Result:** a PyTorch Geometric Data object with `x`, `edge_index`, `edge_attr`, and `y`

11 Troubleshooting

Symptom	Likely cause	Fix
n_supervoxels < 20	GTV too large, region too small	Inspect region_array size. Try n_segments=60. Check CT spacing after resampling.
TypeError: multichannel	scikit-image < 0.20 installed	pip install scikit-image==0.21.0
GTV voxels have non-zero labels	gtv_array not passed to function	Always pass gtv_array. Check call in preprocess_and_save_all().
All labels = 1 (single cluster)	peritumoral_mask is all zeros	Check preprocessing.py. Verify region_array is non-empty before Stage 2.
ValueError: empty region	Zero valid voxels in region	Add check: if ct_in_region.size == 0 raise early with patient ID and shape info.
Very irregular supervoxel shapes	Expected — slic_zero adapts locally	Not a bug. Irregular shapes reflect genuine tissue heterogeneity.

12 References

5. **SLIC algorithm:** Achanta R, Shaji A, Smith K, Lucchi A, Fua P, Süsstrunk S. SLIC superpixels compared to state-of-the-art superpixel methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2012;34(11):2274–2282.
6. **SLIC-zero:** Section 3.2 of the above paper. Also discussed in: Achanta R, Süsstrunk S. Superpixels and polygons using simple non-iterative clustering (SNIC). *IEEE CVPR*. 2017.
7. **RadGraph paper:** Bae J, Mani K, Czerwonka L, Vanison C, Ryu S, Prasanna P. Spatial radiomic graphs for outcome prediction in radiation therapy–treated head and neck squamous cell carcinoma using pretreatment CT. *Radiology: Imaging Cancer*. 2025.
8. **PyRadiomics:** van Griethuysen JJM, Fedorov A, Parmar C, et al. Computational radiomics system to decode the radiographic phenotype. *Cancer Research*. 2017;77(21):e104–e107.
9. **scikit-image:** scikit-image team. skimage.segmentation.slic. <https://scikit-image.org/docs/stable/api/skimage.segmentation.html>